Network Timeouts

connect, read, deadlines and the random 502

Watch the 4-min video

In one line: every outbound call needs a connect, read and pool timeout plus an overall deadline; pass the remaining budget downstream, make inner timeouts shorter than outer ones, and keep the app's keep-alive longer than the load balancer's.

1 Why timeouts matter

A downstream service hangs. It doesn't fail, it just stops answering. Without timeouts, every request that calls it **waits forever**. Threads pile up, connection pools drain, and soon your whole service is down too. Timeouts stop one slow dependency from taking everything else with it.

2 The kinds of timeout

Timeout	Limits how long you wait to...	Typical value
Connect	establish the TCP and TLS connection	1–2 s
Read	receive the *next bytes* of the response (not the whole response)	a little above the dependency's p99
Write	send a large request body	a few seconds
Pool	borrow a connection from the pool	short, e.g. 0.5–1 s
Idle	keep an unused connection open before closing it	shorter than the server's

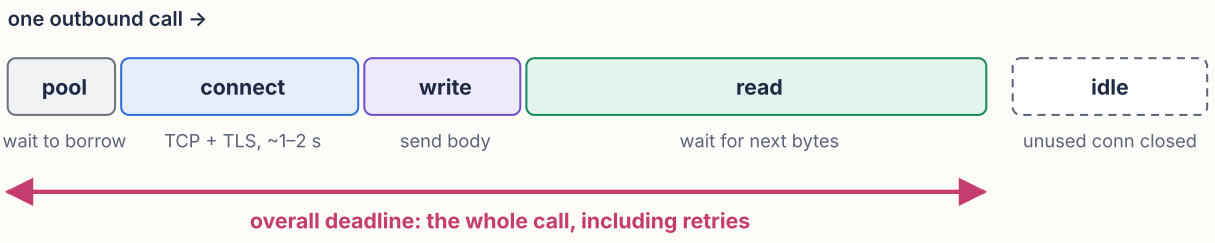
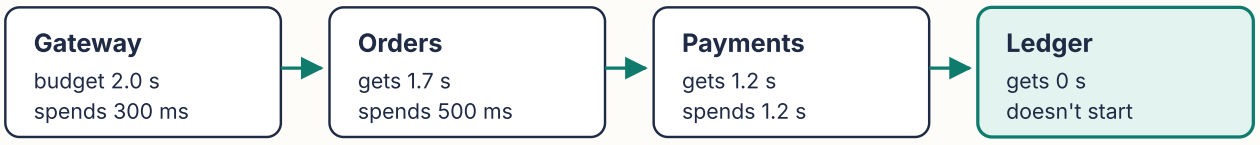


Figure 1. Each phase of a call has its own timeout. The overall deadline caps the sum, including any retries.

3 Per-operation timeouts are not enough

A read timeout of 5 seconds still allows a response that trickles in one byte every 4 seconds, forever: each read finishes in time. So also set a **total deadline** for the whole call, including retries. If the user-facing request has 2 seconds, no single call inside it should be allowed to take 10.

In **microservices, propagate the deadline**. The gateway receives a request with a 2-second budget, spends 300 ms, and passes the remaining 1.7 s to the next service, and so on. gRPC does this natively with deadlines; with HTTP you pass a header. If the budget is already spent, the downstream service doesn't even start the work.



pass the remaining budget down: gRPC deadlines natively, a header (e.g. X-Request-Deadline) over HTTP

if the budget is already spent, the downstream service returns at once instead of doing useless work

Figure 2. Each hop subtracts what it used. By the time the budget reaches zero, nobody downstream wastes effort on an answer no one will wait for.

4 Shrink timeouts inward, and the classic 502

Timeouts should get **shorter as you go deeper**. If the load balancer allows 60 s, the app's request timeout might be 55 s, and the database query timeout 50 s. Then the innermost call fails first with a clear error, and the caller still has time to retry or return a fallback, instead of the outer layer cutting the connection mid-work.

Now the classic bug. The load balancer has a **60-second idle timeout**, but the app server's keep-alive timeout is only **5 seconds** (Uvicorn's default). The app quietly closes an idle connection at the same moment the load balancer reuses it to send a request. The result: random **502 Bad Gateway** errors, often under *light* traffic, when connections sit idle. The fix: make the app's keep-alive longer than the load balancer's, say **75 s**, so the load balancer always closes first.

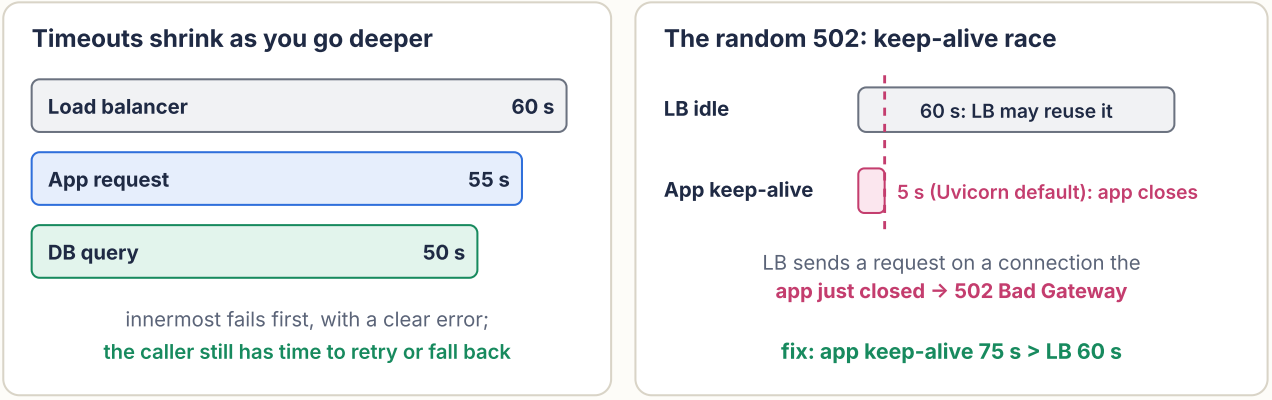


Figure 3. Left: outer layers wait longer than inner ones. Right: when the app closes idle connections before the load balancer does, they race and the loser is a 502.

WORKED EXAMPLE: PRACTICAL DEFAULTS

A payments API has a p99 of 400 ms; your endpoint promises 2 s. Use connect 1 s, read 1 s, pool 0.5 s, and an overall deadline of 1.5 s, leaving 0.5 s for your own work. That allows one retry with backoff, and only because the call carries an idempotency key. In httpx: `httpx.Timeout(1.0, connect=1.0, pool=0.5)` plus `asyncio.timeout(1.5)` around the call. Behind a 60 s load balancer, run `uvicorn --timeout-keep-alive 75`.

Base timeouts on the dependency's **real p99 latency**, not on guesses. Add retries with backoff **only when the operation is idempotent**. And never trust the library default, which is sometimes **no timeout at all**, as with Python's `requests`.

COMMON MISTAKES

- Calling `requests.get(url)` with no `timeout=`: it can wait forever.
- Retries without a total deadline: 3 tries × 5 s read timeout turns a 2 s budget into 15 s.
- An inner timeout longer than the outer one, so the outer layer cuts the work off with no clear error.
- App keep-alive shorter than the load balancer's idle timeout.

5 Key terms

Connect timeout: max time to set up TCP + TLS

Pool timeout: max wait to borrow a connection

Deadline propagation: pass the remaining budget downstream

Read timeout: max gap waiting for the next bytes

Deadline: absolute time budget for the whole call

502 Bad Gateway: proxy got no valid upstream response

6 Interview questions

Q1. What timeouts should every outbound call have? BASIC

A connect timeout (1–2 s), a read timeout (above the dependency's p99), a pool or acquire timeout, and an overall deadline that covers retries.

Q2. Why isn't a read timeout enough on its own? BASIC

It limits the gap between bytes, not the whole response. A server trickling one byte every 4 s never trips a 5 s read timeout, so add a total deadline.

Q3. A dependency hangs and your whole service goes down. Why, and how do you prevent it? SCENARIO

Without timeouts, each request waits forever, holding a thread and a connection; pools drain and everything queues. Set timeouts and a deadline on every call, and add a circuit breaker to fail fast while it is down.

Q4. Your gateway has a 2 s budget and calls three services in a chain. How do you set their timeouts? SCENARIO

Propagate the deadline: each hop gets what's left (2.0 → 1.7 → 1.2 s...), via gRPC deadlines or a header. A service that receives an already-expired budget returns at once without starting work.

Q5. You see random 502s from the load balancer, mostly at night with little traffic. Cause? SCENARIO

A keep-alive race: the LB keeps idle connections for 60 s, the app (Uvicorn's default 5 s) closes them sooner, and the LB reuses one the app just closed. Set the app's keep-alive above the LB's, e.g. 75 s.

Q6. Why should a client timeout be shorter than the server's? TRICKY

So the inner side gives up first. A client that closes idle connections before the server never reuses a dead one, and inner calls fail with a clear error while the caller still has time to retry or fall back.

Q7. Is it safe to retry every timed-out call? TRICKY

No. A timeout doesn't tell you whether the server did the work. Retry only idempotent operations (GET, or POST with an idempotency key), with backoff and jitter, and inside the overall deadline.

Next: SD-C-001 · REST Principles (API Design) backendroadmap.com